

AWK & Shell

Scripting Commands

Complete Reference Guide for Linux / Unix OS

Covers: awk pattern processing • built-in variables • functions • Shell scripting • variables • control flow • functions • I/O • process management

Table of Contents

PART 1 — AWK
1. AWK — Introduction & Basic Syntax
2. AWK — Built-in Variables
3. AWK — Operators
4. AWK — Pattern Types
5. AWK — Actions & Print Commands
6. AWK — String Functions
7. AWK — Arithmetic & Math Functions
8. AWK — Control Flow (if / loops)
9. AWK — Arrays
10. AWK — User-Defined Functions
11. AWK — Input / Output & Pipes
12. AWK — Command-Line Options
13. AWK — Practical Examples
PART 2 — Shell Scripting (Bash)
14. Shell — Script Structure & Basics
15. Shell — Variables & Parameter Expansion

16. Shell — Special Variables
17. Shell — Input / Output & Redirection
18. Shell — Arithmetic
19. Shell — String Operations
20. Shell — Arrays
21. Shell — Conditionals (if / case)
22. Shell — Loops (for / while / until)
23. Shell — Functions
24. Shell — Process & Job Control
25. Shell — File Test Operators
26. Shell — Traps & Signal Handling
27. Shell — Practical Script Examples

PART 1 — AWK (Pattern Scanning & Processing Language)

1. Introduction & Basic Syntax

awk reads input line-by-line, splits each line into fields (\$1, \$2, ...), and executes actions for matching patterns.

General form: `awk 'pattern { action }' file`

Command / Syntax	Description
<code>awk '{ print }' file</code>	Print every line (like cat)
<code>awk '/pat/ { print }' file</code>	Print lines matching pattern
<code>awk '{ print \$1 }' file</code>	Print first field of each line
<code>awk '{ print \$1, \$3 }' file</code>	Print fields 1 and 3
<code>awk '{ print \$NF }' file</code>	Print last field
<code>awk 'NR==5' file</code>	Print line 5 only
<code>awk 'NR>=3 && NR<=7' file</code>	Print lines 3 to 7
<code>awk 'END { print NR }' file</code>	Count total lines
<code>awk -F: '{ print \$1 }' /etc/passwd</code>	Use : as field separator
<code>awk -F',' '{ print \$2 }' data.csv</code>	Use comma as separator (CSV)
<code>awk -v var=5 '{ print \$var }' file</code>	Pass variable from shell
<code>awk 'BEGIN{} /pat/{} END{}' file</code>	Full structure: BEGIN, body, END

2. Built-in Variables

Command / Syntax	Description
<code>\$0</code>	Entire current line
<code>\$1, \$2, ..., \$NF</code>	Fields 1, 2, ... last field
<code>NF</code>	Number of fields in current record
<code>NR</code>	Current record (line) number (total across all files)
<code>FNR</code>	Record number within current file
<code>FS</code>	Input field separator (default: whitespace)
<code>OFS</code>	Output field separator (default: space)
<code>RS</code>	Input record separator (default: newline)
<code>ORS</code>	Output record separator (default: newline)
<code>FILENAME</code>	Name of current input file
<code>ARGC</code>	Number of command-line arguments
<code>ARGV</code>	Array of command-line arguments
<code>ENVIRON</code>	Array of environment variables (ENVIRON["HOME"])
<code>OFMT</code>	Output format for numbers (default: %.6g)

Command / Syntax	Description
CONVFMT	Conversion format for numbers (default: %.6g)
SUBSEP	Subscript separator for multi-dim arrays (default: \034)

3. Operators

Command / Syntax	Description
+ - * / % ^	Arithmetic: add, sub, mul, div, mod, power
++ --	Increment / Decrement
= += -= *= /= %= ^=	Assignment operators
== != < > <= >=	Comparison operators
&& !	Logical AND, OR, NOT
~ !~	Regex match / does not match: \$1 ~ /pat/
? :	Ternary: condition ? val_true : val_false
in	Array membership: if (key in arr)
" " (space)	String concatenation: s = "Hello" " " "World"

4. Pattern Types

Command / Syntax	Description
BEGIN { ... }	Execute before first input line
END { ... }	Execute after last input line
/regex/	Match lines where \$0 ~ /regex/
!/regex/	Match lines NOT matching regex
\$2 ~ /regex/	Match lines where field 2 matches regex
expression	Match if expression is true (non-zero/non-empty)
pat1, pat2	Range: from first match of pat1 to pat2
NR % 2 == 0	Match even-numbered lines
NF > 3	Match lines with more than 3 fields
\$1 == "value"	Match lines where field 1 equals value

5. Actions & Print Commands

Command / Syntax	Description
print	Print \$0 (current line)
print \$1, \$2	Print fields separated by OFS
print \$1 "-" \$2	Print with string concatenation
printf "%s %d\n", \$1, \$2	Formatted print (like C printf)

Command / Syntax	Description
<code>printf "%-10s %5d\n", \$1, \$2</code>	Left-aligned field / right-aligned number
<code>print > "out.txt"</code>	Redirect output to file (overwrites)
<code>print >> "out.txt"</code>	Append output to file
<code>print "cmd"</code>	Pipe output to shell command
<code>print "sort -k2"</code>	Pipe to sort by field 2
<code>close("out.txt")</code>	Close file/pipe (flush buffer)
<code>next</code>	Skip to next input record
<code>exit</code>	Stop processing, jump to END
<code>exit 1</code>	Exit with status code 1
<code>getline</code>	Read next line into \$0
<code>getline var</code>	Read next line into variable
<code>getline < "file"</code>	Read line from file into \$0
<code>"cmd" getline var</code>	Read output of cmd into var

6. String Functions

Command / Syntax	Description
<code>length(s)</code>	Length of string <code>s</code> (or <code>\$0</code> if omitted)
<code>substr(s, i)</code>	Substring from position <code>i</code> to end
<code>substr(s, i, n)</code>	Substring from position <code>i</code> , length <code>n</code>
<code>index(s, t)</code>	Position of string <code>t</code> in <code>s</code> (0 if not found)
<code>split(s, arr)</code>	Split <code>s</code> into array <code>arr</code> using FS; returns count
<code>split(s, arr, sep)</code>	Split <code>s</code> into <code>arr</code> using <code>sep</code> as separator
<code>sub(/pat/, repl)</code>	Replace FIRST match of <code>pat</code> in <code>\$0</code> with <code>repl</code>
<code>sub(/pat/, repl, s)</code>	Replace first match in string <code>s</code>
<code>gsub(/pat/, repl)</code>	Replace ALL matches in <code>\$0</code>
<code>gsub(/pat/, repl, s)</code>	Replace all matches in string <code>s</code>
<code>match(s, /pat/)</code>	Find <code>pat</code> in <code>s</code> ; sets RSTART, RLENGTH
<code>gensub(/pat/, repl, "g")</code>	Return modified string (gawk only)
<code>toupper(s)</code>	Convert string to UPPERCASE
<code>tolower(s)</code>	Convert string to lowercase
<code>sprintf(fmt, ...)</code>	Return formatted string (like printf)
<code>strftime(fmt, ts)</code>	Format timestamp (gawk only)
<code>system()</code>	Return current Unix timestamp (gawk only)

7. Arithmetic & Math Functions

Command / Syntax	Description
<code>int(x)</code>	Truncate <code>x</code> to integer (towards zero)
<code>sqrt(x)</code>	Square root of <code>x</code>
<code>exp(x)</code>	<code>e</code> raised to power <code>x</code>
<code>log(x)</code>	Natural logarithm of <code>x</code>
<code>sin(x)</code>	Sine of <code>x</code> (radians)
<code>cos(x)</code>	Cosine of <code>x</code> (radians)
<code>atan2(y, x)</code>	Arctangent of <code>y/x</code> (radians)
<code>rand()</code>	Random float: $0 \leq r < 1$
<code>srand()</code>	Seed random number generator (current time)
<code>srand(n)</code>	Seed with <code>n</code> for reproducible results

8. Control Flow (if / loops)

Command / Syntax	Description
if (cond) { ... }	If statement
if (cond) { ... } else { ... }	If-else
if (c1) {} else if (c2) {}	If-else if chain
while (cond) { ... }	While loop
do { ... } while (cond)	Do-while loop
for (i=1; i<=n; i++) { ... }	C-style for loop
for (key in arr) { ... }	Iterate over array keys
break	Exit loop
continue	Skip to next iteration
next	Skip to next input record (like continue for awk)
exit	Stop processing, go to END block

9. Arrays

Command / Syntax	Description
arr[key] = val	Assign value to array element
print arr[key]	Access array element
if (key in arr)	Test if key exists in array
delete arr[key]	Delete one element
delete arr	Delete entire array
for (k in arr) print k	Iterate over all keys
arr[k1][k2] = val	Multi-dimensional array (gawk)
arr[k1,k2] = val	Simulated 2D array using SUBSEP
split(s, arr)	Populate array from split string
asorti(arr, dest)	Sort array indices into dest (gawk)
asort(arr)	Sort array values in-place (gawk)

10. User-Defined Functions

Command / Syntax	Description
function name(p1,p2) { ... }	Define a function
function name(p1, lv) { ... }	Extra params become local variables
return value	Return a value
name(arg1, arg2)	Call a function
function max(a,b){return a>b?a:b}	Example: max function

11. Input / Output & Pipes

Command / Syntax	Description
<code>awk ... file1 file2</code>	Process multiple files sequentially
<code>cmd awk '...'</code>	Pipe command output into awk
<code>awk '...' cmd</code>	Pipe awk output to command
<code>print > "file"</code>	Write to file (once per execution)
<code>print >> "file"</code>	Append to file
<code>print "sort"</code>	Pipe to sort (pipe stays open)
<code>close("sort")</code>	Close pipe, flush sort output
<code>getline line < "/etc/hostname"</code>	Read from file into variable
<code>while ((getline line < f) > 0)</code>	Read file line by line
<code>system("ls -l")</code>	Run shell command, return exit code

12. Command-Line Options

Command / Syntax	Description
<code>awk -F sep</code>	Set input field separator
<code>awk -v var=val</code>	Set variable before execution
<code>awk -f script.awk</code>	Read awk program from file
<code>awk -W compat</code>	POSIX compatibility mode
<code>gawk -M</code>	Enable arbitrary precision arithmetic
<code>gawk --lint</code>	Warn about non-portable constructs
<code>gawk -i inplace</code>	Edit file in-place (gawk 4.1+)
<code>gawk -E file</code>	Execute file as awk program (disable -v etc.)

13. Practical AWK Examples

Command / Syntax	Description
<code>awk 'NR%2==0' file</code>	Print even-numbered lines
<code>awk '{sum+=\$1} END{print sum}' file</code>	Sum first column
<code>awk '{sum+=\$1} END{print sum/NR}' file</code>	Average first column
<code>awk '!seen[\$0]++' file</code>	Remove duplicate lines (preserve order)
<code>awk 'length(\$0)>80' file</code>	Print lines longer than 80 chars
<code>awk '{print NR": "\$0}' file</code>	Number every line
<code>awk '\$3>100{print \$1,\$3}' file</code>	Filter rows where col 3 > 100
<code>awk -F: '{print \$1}' /etc/passwd</code>	List all usernames
<code>awk 'NR==1{next} {print}' file</code>	Skip header line
<code>awk '{print \$NF}' file</code>	Print last field of each line
<code>awk '{gsub(/foo/, "bar")} 1' file</code>	Replace foo with bar everywhere
<code>awk 'BEGIN{OFS=","} {\$1=\$1} 1' f</code>	Reformat with comma OFS
<code>awk '{a[\$1]+=\$2} END{for(k in a) print k, a[k]}' file</code>	Calculate sum by key, collapse
<code>awk 'FNR==NR{a[\$1];next} \$1 in a' f1 f2</code>	Find common lines (join)
<code>awk '{print "sort"}' file</code>	Sort output via pipe
<code>awk 'BEGIN{print "Start"} {print} END{print "End"}'</code>	Add header/footer

PART 2 — Shell Scripting (Bash)

14. Script Structure & Basics

A shell script is a plain text file with executable permissions. The shebang line tells the OS which interpreter to use.

Command / Syntax	Description
<code>#!/bin/bash</code>	Shebang — use bash interpreter
<code>#!/usr/bin/env bash</code>	Portable shebang (finds bash in PATH)
<code>#!/bin/sh</code>	POSIX sh (more portable, fewer features)
<code>chmod +x script.sh</code>	Make script executable
<code>./script.sh</code>	Run script from current directory
<code>bash script.sh</code>	Run without execute permission
<code>bash -x script.sh</code>	Debug: trace each command executed
<code>bash -n script.sh</code>	Check syntax without running
<code>bash -e script.sh</code>	Exit on first error
<code>set -e</code>	Exit immediately on error (inside script)
<code>set -u</code>	Treat unset variables as error
<code>set -o pipefail</code>	Fail if any command in pipe fails
<code>set -euo pipefail</code>	Common strict mode combination
<code># comment</code>	Single-line comment
<code>source file</code> or <code>. file</code>	Execute file in current shell context

15. Variables & Parameter Expansion

Command / Syntax	Description
<code>var=value</code>	Assign variable (no spaces around =)
<code>echo \$var</code> or <code>\${var}</code>	Use variable (braces recommended)
<code>readonly var=value</code>	Declare read-only variable
<code>unset var</code>	Remove variable
<code>export var</code>	Export to child processes (environment)
<code>local var=val</code>	Local variable inside function
<code>declare -i n=5</code>	Declare integer variable
<code>declare -r const=10</code>	Declare read-only (like readonly)
<code>declare -l str</code>	Auto-lowercase string
<code>declare -u str</code>	Auto-uppercase string
<code>\${var:-default}</code>	Use default if var unset or empty

Command / Syntax	Description
<code>\${var:=default}</code>	Assign default if var unset or empty
<code>\${var:+alt}</code>	Use alt if var IS set (else empty)
<code>\${var:?msg}</code>	Error with msg if var unset or empty
<code>\${#var}</code>	Length of string in var
<code>\${var:offset}</code>	Substring from offset
<code>\${var:offset:len}</code>	Substring: offset, length len
<code>\${var#pattern}</code>	Remove shortest prefix matching pattern
<code>\${var##pattern}</code>	Remove longest prefix matching pattern
<code>\${var%pattern}</code>	Remove shortest suffix matching pattern
<code>\${var%%pattern}</code>	Remove longest suffix matching pattern
<code>\${var/old/new}</code>	Replace first occurrence of old with new
<code>\${var//old/new}</code>	Replace all occurrences
<code>\${var^}</code>	Uppercase first character
<code>\${var^^}</code>	Uppercase all characters
<code>\${var,}</code>	Lowercase first character
<code>\${var,,}</code>	Lowercase all characters

16. Special Variables

Command / Syntax	Description
\$0	Script name (path as invoked)
\$1 ... \$9	Positional parameters (arguments)
\${10}	10th argument (braces required for 10+)
\$#	Number of arguments passed
\$@	All arguments as separate words (quoted: "\$@")
\$*	All arguments as single word
\$\$	PID of current shell / script
\$_	PID of last background command
\$?	Exit status of last command (0 = success)
\$-	Current shell option flags
_	Last argument of previous command
BASH_VERSION	Bash version string
BASH_PID	PID of current bash process
LINENO	Current line number in script
FUNCNAME	Array of function call stack names
SECONDS	Seconds elapsed since shell started
RANDOM	Random integer 0–32767 (changes each access)
HOSTNAME	Hostname of the machine
PWD	Current working directory
HOME	Home directory of current user
PATH	Executable search path
IFS	Internal field separator (default: space/tab/newline)
OLDPWD	Previous working directory

17. Input / Output & Redirection

Command / Syntax	Description
echo "text"	Print text to stdout
echo -n "text"	Print without newline
echo -e "a\tb"	Interpret escape sequences
printf "%s\n" val	Formatted print
read var	Read one line from stdin into var
read -p "Prompt: " var	Prompt and read
read -s var	Read silently (no echo — for passwords)

Command / Syntax	Description
<code>read -t 5 var</code>	Read with 5-second timeout
<code>read -a arr</code>	Read words into array
<code>read -r var</code>	Raw mode (backslash not escape)
<code>cmd > file</code>	Redirect stdout to file (overwrite)
<code>cmd >> file</code>	Redirect stdout to file (append)
<code>cmd 2> err.txt</code>	Redirect stderr to file
<code>cmd 2>&1</code>	Redirect stderr to stdout
<code>cmd > out.txt 2>&1</code>	Both stdout and stderr to file
<code>cmd &> file</code>	Both stdout and stderr (bash shorthand)
<code>cmd < file</code>	Read stdin from file
<code>cmd1 cmd2</code>	Pipe stdout of cmd1 to stdin of cmd2
<code>cmd1 tee file cmd2</code>	Pipe and save copy to file
<code>cmd <<EOF\ntext\nEOF</code>	Here-doc: multi-line stdin
<code>cmd <<< "string"</code>	Here-string: single string as stdin
<code>exec 3> file</code>	Open file descriptor 3 for writing
<code>echo "x" >&3</code>	Write to file descriptor 3
<code>exec 3>&-</code>	Close file descriptor 3
<code>/dev/null</code>	Discard output: <code>cmd > /dev/null 2>&1</code>

18. Arithmetic

Command / Syntax	Description
<code>\$((expr))</code>	Arithmetic expansion: <code>result=\$((3+4))</code>
<code>((expr))</code>	Arithmetic evaluation (no \$, used in if/while)
<code>let expr</code>	Evaluate arithmetic: <code>let n=a+b</code>
<code>expr 3 + 4</code>	External <code>expr</code> command (older, slower)
<code>\$((a + b))</code>	Addition
<code>\$((a - b))</code>	Subtraction
<code>\$((a * b))</code>	Multiplication
<code>\$((a / b))</code>	Integer division
<code>\$((a % b))</code>	Modulo
<code>\$((a ** b))</code>	Exponentiation (bash 3+)
<code>\$((a++))</code>	Post-increment
<code>\$((++a))</code>	Pre-increment
<code>\$((a & b))</code>	Bitwise AND
<code>\$((a b))</code>	Bitwise OR
<code>\$((a ^ b))</code>	Bitwise XOR
<code>\$((a << n))</code>	Left shift
<code>\$((a >> n))</code>	Right shift
<code>bc <<< "3.14*2"</code>	Floating-point arithmetic via <code>bc</code>
<code>awk 'BEGIN{print 1/3}'</code>	Floating-point via <code>awk</code>

19. String Operations

Command / Syntax	Description
<code>str="Hello World"</code>	Assign string
<code>\${#str}</code>	String length
<code>\${str:6}</code>	Substring from index 6
<code>\${str:6:5}</code>	Substring: start 6, length 5
<code>\${str,,}</code>	Convert to lowercase
<code>\${str^^}</code>	Convert to uppercase
<code>\${str/World/Bash}</code>	Replace first occurrence
<code>\${str//l/L}</code>	Replace all occurrences
<code>\${str#Hello }</code>	Remove prefix 'Hello '
<code>\${str%World}</code>	Remove suffix 'World'
<code>[[\$str == *world*]]</code>	Glob match (contains World)

Command / Syntax	Description
<code>[[\$str =~ ^Hello]]</code>	Regex match (starts with Hello)
<code>s1="Hello"; s2="World"; echo "\${s1}\${s2}"</code>	Concatenation
<code>IFS=',' read -ra arr <<< "a,b,c"</code>	Split string to array by comma

20. Arrays

Command / Syntax	Description
<code>arr=(a b c d)</code>	Declare indexed array
<code>arr[0]=val</code>	Set element at index 0
<code>echo \${arr[0]}</code>	Access element at index 0
<code>echo \${arr[@]}</code>	All elements (separate words)
<code>echo \${arr[*]}</code>	All elements (single word)
<code>echo \${#arr[@]}</code>	Number of elements
<code>echo \${!arr[@]}</code>	All indices
<code>arr+=(e f)</code>	Append elements
<code>unset arr[2]</code>	Delete element at index 2
<code>unset arr</code>	Delete entire array
<code>arr2=("\${arr[@]}")</code>	Copy array
<code>slice=("\${arr[@]:1:2}")</code>	Slice: start 1, length 2
<code>declare -A map</code>	Declare associative array (dict)
<code>map[key]=value</code>	Set key-value pair
<code>echo \${map[key]}</code>	Get value by key
<code>echo \${!map[@]}</code>	All keys
<code>echo \${map[@]}</code>	All values
<code>if [[-v map[key]]]</code>	Check if key exists (bash 4.2+)
<code>for v in "\${arr[@]}"</code>	Iterate over array elements

21. Conditionals (if / case)

Command / Syntax	Description
<code>if [cond]; then ... fi</code>	Basic if (POSIX)
<code>if [[cond]]; then ... fi</code>	Extended if (bash)
<code>if cmd; then ... fi</code>	if based on command exit code
<code>if [c]; then ...; else ...; fi</code>	if-else
<code>if [c1]; then ...; elif [c2]; then ...; else ...; fi</code>	if-elif-else
<code>[a = b]</code>	String equality (POSIX)
<code>[a != b]</code>	String inequality
<code>[[a == b]]</code>	String equality (bash)
<code>[[str =~ regex]]</code>	Regex match (bash)
<code>[-z "\$str"]</code>	True if string is empty
<code>[-n "\$str"]</code>	True if string is non-empty
<code>[\$a -eq \$b]</code>	Integer equal
<code>[\$a -ne \$b]</code>	Integer not equal
<code>[\$a -lt \$b]</code>	Integer less than
<code>[\$a -le \$b]</code>	Integer less than or equal
<code>[\$a -gt \$b]</code>	Integer greater than
<code>[\$a -ge \$b]</code>	Integer greater than or equal
<code>((a > b))</code>	Arithmetic comparison
<code>[cond1] && [cond2]</code>	AND (POSIX style)
<code>[[cond1 && cond2]]</code>	AND (bash style)
<code>[[cond1 cond2]]</code>	OR (bash style)
<code>case \$var in\n pat) cmd;;\nesac</code>	Case statement
<code>case \$var in\n a b) cmd;;\n *) defaultcmd;;\nesac</code>	Case with default

22. Loops (for / while / until)

Command / Syntax	Description
<code>for var in list; do ... done</code>	For-in loop over list
<code>for i in 1 2 3 4 5; do ... done</code>	Loop over explicit values
<code>for i in {1..10}; do ... done</code>	Loop over brace expansion range
<code>for i in {0..20..2}; do ... done</code>	Brace range with step 2
<code>for ((i=0; i<10; i++)); do ... done</code>	C-style for loop
<code>for f in *.txt; do ... done</code>	Loop over files
<code>for line in \$(cat file); do ... done</code>	Loop over file lines (field-split)

Command / Syntax	Description
<code>while IFS= read -r line; do ... done < file</code>	Safe loop over file lines
<code>while [cond]; do ... done</code>	While loop
<code>while ((i < 10)); do ... done</code>	While with arithmetic
<code>until [cond]; do ... done</code>	Until loop (runs until true)
<code>break</code>	Exit loop
<code>break 2</code>	Break out of 2 nested loops
<code>continue</code>	Skip to next iteration
<code>continue 2</code>	Continue outer loop
<code>select opt in a b c; do ... done</code>	Interactive menu loop

23. Functions

Command / Syntax	Description
<code>function name() { ... }</code>	Declare function (bash style)
<code>name() { ... }</code>	Declare function (POSIX style)
<code>name</code>	Call function (no parentheses)
<code>name arg1 arg2</code>	Call with arguments
<code>\$1, \$2, ...</code>	Access function arguments inside function
<code>return 0</code>	Return exit status (0=success)
<code>return 1</code>	Return failure status
<code>local var=val</code>	Local variable (invisible outside)
<code>echo "result"</code>	Return value via stdout capture
<code>result=\$(name arg)</code>	Capture function output
<code>declare -f name</code>	Show function definition
<code>declare -F</code>	List all defined function names
<code>unset -f name</code>	Remove function definition
<code>name() { ;; }</code>	No-op function (: is null command)
<code>trap name EXIT</code>	Run function on script exit

24. Process & Job Control

Command / Syntax	Description
<code>cmd &</code>	Run cmd in background
<code>jobs</code>	List background/stopped jobs
<code>fg</code>	Bring last background job to foreground
<code>fg %n</code>	Bring job n to foreground
<code>bg %n</code>	Resume job n in background
<code>wait</code>	Wait for all background jobs to finish
<code>wait \$pid</code>	Wait for specific PID
<code>kill PID</code>	Send SIGTERM to process
<code>kill -9 PID</code>	Send SIGKILL (force kill)
<code>kill -l</code>	List signal names
<code>killall name</code>	Kill all processes named name
<code>pkill pattern</code>	Kill processes by name pattern
<code>\$(cmd)</code>	Command substitution (capture output)
<code>`cmd`</code>	Command substitution (legacy syntax)
<code>(cmd1; cmd2)</code>	Run commands in subshell

Command / Syntax	Description
{ cmd1; cmd2; }	Run commands in current shell (group)
cmd1 && cmd2	Run cmd2 only if cmd1 succeeds
cmd1 cmd2	Run cmd2 only if cmd1 fails
cmd1 ; cmd2	Run cmd2 regardless of cmd1 result
nohup cmd &	Run cmd immune to hangup signal
disown %n	Remove job from job table
time cmd	Time how long cmd takes
nice -n 10 cmd	Run cmd with lower priority
sleep n	Pause for n seconds

25. File Test Operators

Command / Syntax	Description
[-e file]	True if file/directory exists
[-f file]	True if regular file exists
[-d dir]	True if directory exists
[-L file]	True if symbolic link
[-r file]	True if file is readable
[-w file]	True if file is writable
[-x file]	True if file is executable
[-s file]	True if file size is > 0 (non-empty)
[-z file]	True if file has zero size (note: -z for strings)
[-b file]	True if block device
[-c file]	True if character device
[-p file]	True if named pipe (FIFO)
[-S file]	True if socket
[-t fd]	True if file descriptor fd is open on terminal
[-N file]	True if file modified since last read
[-O file]	True if file owned by current user
[-G file]	True if file group matches current group
[f1 -nt f2]	True if f1 is newer than f2
[f1 -ot f2]	True if f1 is older than f2
[f1 -ef f2]	True if f1 and f2 are same file (hard link)

26. Traps & Signal Handling

Command / Syntax	Description
trap 'cmd' SIGNAL	Run cmd when signal received
trap 'cleanup' EXIT	Run cleanup on any script exit
trap 'handler' INT	Catch Ctrl+C (SIGINT)
trap 'handler' TERM	Catch SIGTERM (kill default)
trap 'handler' ERR	Catch any command error (with set -e)
trap '' INT	Ignore SIGINT
trap - INT	Reset signal to default behavior
trap -p INT	Show current trap for signal
kill -INT \$\$	Send SIGINT to current script
Signal: 1 = HUP	Hang Up

Command / Syntax	Description
Signal: 2 = INT	Interrupt (Ctrl+C)
Signal: 9 = KILL	Force kill (cannot be caught)
Signal: 15 = TERM	Terminate (default kill)
Signal: 17 = CHLD	Child process stopped/terminated

27. Practical Shell Script Examples

Command / Syntax	Description
for f in *.log; do gzip "\$f"; done	Gzip all .log files
find . -name '*.txt' xargs grep 'ERROR'	Grep across many files
while read line; do echo "\$line"; done < file.txt	Process file line by line
ls sort uniq -c sort -rn	Count & rank file names
du -sh */ sort -rh head -10	Top 10 largest directories
if [[! -d "\$dir"]]; then mkdir -p "\$dir"; fi	Create dir if not exists
timestamp=\$(date +%Y%m%d_%H%M%S)	Generate timestamp string
trap 'rm -f /tmp/myfile.\$\$' EXIT	Cleanup temp file on exit
result=\$(grep -c 'pattern' file)	Count matching lines into var
[[\$# -ne 2]] && echo "Usage: \$0 a b" && exit 1	Argument count check
read -p "Are you sure? [y/N] " ans; [[\$ans == n]] && exit 1	Yes/No prompt
exec >> /var/log/my.log 2>&1	Redirect all output to log
for pid in \$(pgrep python); do kill \$pid; done	Kill all Python processes
arr=(\$(ls *.sh)); echo "\${#arr[@]} scripts"	Count shell scripts
diff <(sort file1) <(sort file2)	Compare sorted files
paste -d, col1.txt col2.txt > out.csv	Merge files as CSV columns
cut -d: -f1,3 /etc/passwd	Extract fields 1 and 3
tee log.txt mail -s 'Report' user@host	Log and email output

Notes: `awk` — `gawk` (GNU `awk`) supports extra features like `gensub()`, `systime()`, `-i` inplace; POSIX `awk` may not. Shell — commands marked as 'bash' require Bash 3+; Bash 4+ adds associative arrays and improved string ops. Use 'man `awk`', 'man `bash`' for full documentation.